Deep Reinforcement Learning

Advanced Algorithms (Part II)

Prof. Dr. Sebastian Peitz

Scientific Machine Learning group @ Paderborn University

Winter term 2026/27

PADERBORN UNIVERSITY

Content

Content

- The evolution of modern RL algorithms
- Part (II): Improving Q-learning
  - Deterministic policy gradient
  - Deep deterministic policy gradient (DDPG)
  - TD3
  - Soft Actor-Critic

Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6	Brief introduction to deep learning	The basics for what comes next
7	Value function approximation	Value estimation with function approximation
8	Deep Q-learning	Q-learning with neural networks
9	Policy gradients	Direct optimization of the policy
10	Actor-critic algorithms	Improved policy gradients via value functions
11	Advanced algorithms (Part I): From policy gradient to PPO	The PG route to modern RL algorithms
12	Advanced algorithms (Part II): From Q-learning to Soft Actor-Critic	The AC route to modern RL algorithms
13	Exploration	
	Model-Based Control	
	Advanced Topics	

Lecture contents

The evolution of modern RL algorithms

The evolution of modern RL algorithms

(I) Policy gradient methods

- Policy is explicitly parameterized and optimized directly:

$$L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)}\left[r(\tau)\right].$$

- Gradient ascent: $\phi \leftarrow \phi + \alpha \nabla_\phi L_\pi(\phi)$.
- Methods are typically **on-policy**.
- **Algorithms**: REINFORCE → Actor-Critic → Natural Policy Gradient → Trust-region policy optimization (TRPO) → Proximal policy optimization (PPO).

Shortcomings

- High variance gradients.
- Unstable policy updates.
- Catastrophic performance collapse.

Improvement strategy

- How to safely update policies.

(II) Value-based methods

- Approximation of the Q-function:

$$Q^*(s,a) = r + \max_{a' \in \mathcal{A}} Q^*(s',a').$$

- Extract implicit policy: $\pi(s) = \arg\max_{a \in \mathcal{A}} Q^*(s,a)$.
- Typically **off-policy**.
- **Algorithms**: Q-learning → DQN → Deep deterministic policy gradient (DDPG) → TD3 → Soft actor-critic (SAC).

Shortcomings

- Instability from bootstrapping.
- Overestimation bias.
- Maximization over actions / continuous action spaces.

Improvement strategy

- Stabilizing Q-learning with function approximation.
- Solving the continuous argmax problem.

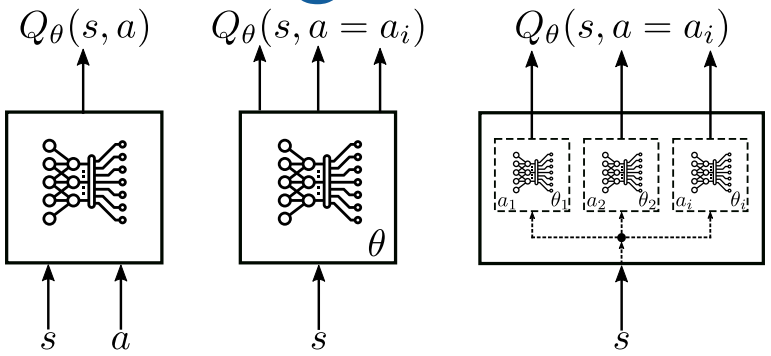
Part (II): Improving Q-learning

The problem of continuous actions in Q-learning

- In a standard DQN, the optimal policy is implicit.
- To choose the best action a in a given state s, the agent evaluates the Q-values for all possible actions and picks the one that maximizes the expected return:

$$\pi(s) = \arg\max_a Q^*(s, a)$$

- Works well for discrete action spaces (e.g., Left, Right, Jump).



Types of architectures (Abdelwanis et al. 2026)

- For continuous $\mathcal{A}$ (e.g., controlling the torque of a robotic joint), we have an infinite number of possible actions.
- To find the absolute maximum of the Q-function in this scenario, we can pursue two options (both very expensive):
  - **Discretize the action space**. For instance, if we have 7 joints and discretize each into just 10 levels, we get 10^7 possible actions.
    $\Rightarrow$ The *"curse of dimensionality"* makes this computationally impossible to solve in real-time.
  - **Optimization**. An iterative optimization algorithm (e.g., gradient ascent) inside the environment loop to find the maximizing a before each step.

Approach: Merging DQN and Actor-Critic

Instead of maximizing over Q, we introduce a function $a = \mu_\phi(s)$ such that $Q^*(s, \mu_\phi(s)) \approx \max_a Q^*(s, a)$.

Challenges:

- We now have to optimize for the policy directly $\Rightarrow$ policy gradient!
- In off-policy algorithms like Q-learning, the variance becomes an even bigger problem!
  - Off-policy policy gradient.

- Deterministic policies with reduced variance.

Recall: Policy gradient and actor-critic

Policy gradient theorem – formulation via reward trajectories (sampling version in blue)

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_t|s_t)\left(\sum_{t'=t}^{T-1} r_{t'}\right)\right] \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t'=t}^{T-1}\nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})\left(\sum_{t'=t}^{T-1} r_{i,t'}\right)\right).$$

Policy gradient theorem – formulation using the Q-function and the Actor-Critic architecture

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_t|s_t) A_\theta(s_t,a_t)\right] \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) A_\theta(s_{i,t},a_{i,t})\right).$$

Main drawbacks

1. **High-variance** gradient.
2. **Inefficient for continuous actions**, since sampling in high-dimensional settings becomes inefficient.
3. **On-policy** $\Rightarrow$ sample-inefficient (importance sampling makes problem 1. even worse!)

Approach: Find a more sample-efficient and off-policy capable version $\Rightarrow$ deterministic policy!

Off-policy policy gradients (1)

- (Degris, White, and Sutton 2012) presented an alternative form for the off-policy policy gradient (for finite $\mathcal{S}$ and $\mathcal{A}$) using an approximation. The starting point is the definition of the value function:

$$V^{\pi_\phi} = \sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} \pi_\phi(a|s) Q^{\pi_\phi}(s,a),$$

  where β is some behavior policy and $\rho_\beta(s) = \lim_{t \to \infty} p(s|s_0, \beta)$ is the limiting distribution of states when following β.

- Then, taking the gradient of V^{π_ϕ} with respect to ϕ, we get (via the product rule of differentiation):

$$\nabla_\phi V^{\pi_\phi} = \nabla_\phi \left[\sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} \pi_\phi(a|s) Q^{\pi_\phi}(s,a) \right] = \sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} [\nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a) + \pi_\phi(a|s) \nabla_\phi Q^{\pi_\phi}(s,a)].$$

- Ignoring the second part (i.e., ~~$\pi_\phi(a|s) \nabla_\phi Q^{\pi_\phi}(s,a)$~~ ; justification details in (Degris, White, and Sutton 2012)), we get

$$\nabla_\phi V^{\pi_\phi} \approx \sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} \nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a) = \mathbb{E}_{s \sim \rho_\beta} \left[\sum_{a \in \mathcal{A}} \nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a) \right].$$

💡 In contrast to the on-policy case (see the Actor-Critic lecture), here it is not possible to formulate a recursive formula which would allow us to find a closed-form statement for $\nabla_\phi Q^{\pi_\phi}(s,a)$ via $\nabla_\phi V^{\pi_\phi}(s)$. This only works in the on-policy setting!

Off-policy policy gradients (2)

$$\nabla_\phi V^{\pi_\phi} \approx \sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} \nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a) = \mathbb{E}_{s \sim \rho_\beta} \left[\sum_{a \in \mathcal{A}} \nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a) \right].$$

- To make this off-policy w.r.t. the actions as well, we introduce *importance sampling* for β:

$$\nabla_\phi V^{\pi_\phi} \approx \sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} \beta(a|s) \underbrace{\frac{\pi_\phi(a|s)}{\beta(a|s)}}_{=\kappa_\phi(s,a)} \frac{\nabla_\phi \pi_\phi(a|s)}{\pi_\phi(a|s)} Q^{\pi_\phi}(s,a) = \sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} \beta(a|s) \kappa_\phi(s,a) \nabla_\phi \log \pi_\phi(a|s) Q^{\pi_\phi}(s,a)$$

$$= \mathbb{E}_{s \sim \rho_\beta, a \sim \beta(\cdot|s)} \left[\kappa_\phi(s,a) \nabla_\phi \log \pi_\phi(a|s) Q^{\pi_\phi}(s,a) \right].$$

- The starting point in (Silver et al. 2014) was very similar, but using continuous state and action spaces:

$$\boxed{\nabla_\phi V^{\pi_\phi} \approx \int_{\mathcal{S}} \rho_\beta(s) \int_{\mathcal{A}} \kappa_\phi(s,a) \nabla_\phi \log \pi_\phi(a|s) Q^{\pi_\phi}(s,a) \, \mathrm{d}a \, \mathrm{d}s = \mathbb{E}_{s \sim \rho_\beta, a \sim \beta(\cdot|s)} \left[\kappa_\phi(s,a) \nabla_\phi \log \pi_\phi(a|s) Q^{\pi_\phi}(s,a) \right].}$$

- We now have derived a simplified formula for the off-policy policy gradient.
- Aside from the approximation we just made, it's similar to the on-policy policy gradient!
- However, we still suffer from the large variance issue.

Deterministic policy gradient

Deterministic off-policy policy gradients

- Drop the randomness from the policy μ such that it becomes a **deterministic function**: $a = \mu_\phi(s)$.
- Reformulate V via Q: no need for expecations (i.e., sum / integrate over $\mathcal{A}$): $V^{\mu_\phi}(s) = Q^{\mu_\phi}(s, \mu_\phi(s))$.
- Deterministic: we are incapable of exploration! The theorem is only practically useful if we sample from off-policy data:

$$L_\pi(\phi) = \mathbb{E}_{s \sim \rho_\beta}\left[Q^{\mu_\phi}(s, \mu_\phi(s))\right] = \int_{\mathcal{S}} \rho_\beta(s) Q^{\mu_\phi}(s, \mu_\phi(s)) \, \mathrm{d}s.$$

Deterministic policy gradient theorem (Silver et al. 2014)

Exact version: on-policy ($\rho_\beta = \rho_{\mu_\phi}$)

$$\begin{aligned} \nabla_\phi L_\pi(\phi) &= \nabla_\phi \left[\int_{\mathcal{S}} \rho_\beta(s) Q^{\mu_\phi}(s, \mu_\phi(s)) \, \mathrm{d}s\right] = \int_{\mathcal{S}} \rho_\beta(s) \nabla_a Q^{\mu_\phi}(s, \mu_\phi(s))\big|_{a=\mu_\phi(s)} \nabla_\phi \mu_\phi(s) \, \mathrm{d}s \\ &= \mathbb{E}_{s \sim \rho_\beta}\left[\nabla_a Q^{\mu_\phi}(s, \mu_\phi(s))\big|_{a=\mu_\phi(s)} \nabla_\phi \mu_\phi(s)\right]. \end{aligned} \tag{1}$$

Approximate version: off-policy ($\rho_\beta \neq \rho_{\mu_\phi}$, and we use the same simplification as earlier, i.e., ~~$\mu_\phi(a|s) \nabla_a Q^{\mu_\phi}(s, a)$~~)

$$\nabla_\phi L_\pi(\phi) \approx \mathbb{E}_{s \sim \rho_\beta}\left[\nabla_a Q^{\mu_\phi}(s, \mu_\phi(s))\big|_{a=\mu_\phi(s)} \nabla_\phi \mu_\phi(s)\right]. \tag{2}$$

💡 (1) is the limit case of the stochastic policy gradient theorem (i.e., $\mathbf{var}\left[\pi\right] \to 0$) (Silver et al. 2014, Theorem 2).

On-policy deterministic actor-critic

The simplest algorithm we can derive from this: SARSA-type on-policy Actor-Critic:

Algorithm: On-policy deterministic actor-critic

1. Sample $\{s_i, a_i, r_i, s'_i, a'_i\}_{i=1}^{N}$ using $a = \mu_\phi(s)$.
2. TD error:
$$\delta_i = r_i + \gamma Q_\theta(s'_i, a'_i) - Q_\theta(s_i, a_i).$$
3. Semi-gradient Q-function update:
$$\theta \leftarrow \theta + \alpha_\theta \frac{1}{N} \sum_{i=1}^{N} \delta_i \nabla_\theta Q_\theta(s_i, a_i).$$
4. Update policy μ_ϕ by sampling (1):
$$\phi \leftarrow \phi + \alpha \frac{1}{N} \sum_{i=1}^{N} \nabla_a Q_\theta(s_i, a_i) \nabla_\phi \mu_\phi(s_i).$$

Summary of the concept

To update a deterministic policy off-policy, your algorithm does this for a batch of states from the replay buffer:

1. Pass state s into the actor: $a = \mu_\phi(s)$.
2. Pass s and a into the critic network $Q_\theta(s, a)$.
3. Compute how the critic's output changes with respect to that action ($\nabla_a Q$).
4. Compute how the actor's weights change to produce that action change ($\nabla_\phi \mu_\phi$).
5. Multiply them together to update the actor.

Deep deterministic policy gradient (DDPG)

Deep deterministic policy gradient (DDPG)

Problems with vanilla DPG

The original algorithm assumed:

- tabular / linear approximators ("Compatible Function Approximation" in (Silver et al. 2014)),
- stable Q estimation.

When neural networks are used, several problems appear:

- Bootstrapping instability.
- Correlated samples from trajectories.
- Targets change too quickly.
- Poor exploration due to deterministic policy.

Deep deterministic policy gradient (DDPG) (Lillicrap et al. 2016)

addresses these by importing techniques from Deep Q-Networks (DQN).

Core components:

- Actor network $\mu_\phi(s)$
- Critic network $Q_\theta(s, a)$
- Target actor $\mu_{\bar{\phi}}$
- Target critic $Q_{\bar{\theta}}$
- Replay buffer

The DDPG algorithm

1. **Interact**: Sample $\{s_t, a_t, r_t, s_{t+1}\}$ using $a_t = \mu_\phi(s_t)$ (💡 *plus noise for exploration!*) and store in the replay buffer $\mathcal{D}$.
2. **Sample**: Draw random mini-batch of N transitions: $\mathcal{B} \subset \mathcal{D}$.
3. **Update critic**: Calculate the target y_i using the target networks (💡 *Optimal action* $\mu_{\bar{\phi}}(s) \Rightarrow Q$*-learning!*):

$$y_i = r_i + \gamma Q_{\bar{\theta}}(s_{i+1}, \mu_{\bar{\phi}}(s_{i+1})).$$

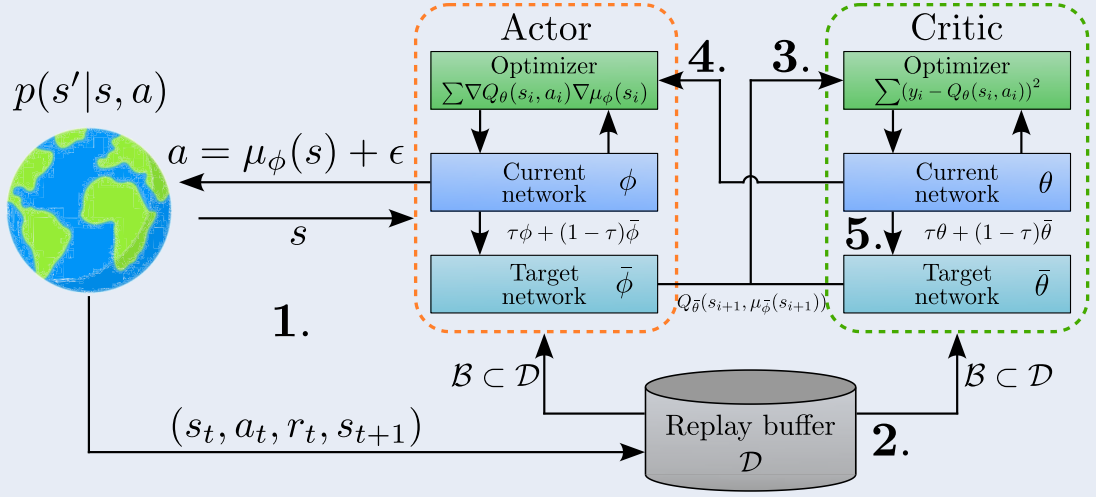
   The *current critic* is updated by minimizing the Bellman error:

$$L_Q(\theta) = \frac{1}{N}\sum_i (y_i - Q_\theta(s_i, a_i))^2, \qquad \theta \leftarrow \theta + \alpha_\theta \frac{2}{N}\sum_{i=1}^{N} (y_i - Q_\theta(s_i, a_i))\nabla_\theta Q_\theta(s_i, a_i).$$

4. **Update actor**: The *current actor* is updated using the sampled deterministic policy gradient (Eq. (1)):

$$\phi \leftarrow \phi + \alpha \frac{1}{N}\sum_{i=1}^{N} \nabla_a Q_\theta(s_i, a)\Big|_{a=\mu_\phi(s_i)} \nabla_\phi \mu_\phi(s_i).$$

5. **Soft updates**: Incremental target updates ($\bar{\phi}$ / $\bar{\theta}$).



The four main changes over DPG

1. **Integration of deep neural networks (CNNs)**
2. **Introduction of the replay buffer**

- Larger datasets to train from.
- Allows for i.i.d. sampling / breaks the temporal correlation of data.

3. **Explicit action noise for exploration**

- Deterministic actor in DPG: it cannot explore on its own.
- DDPG adds an explicit random process $\mathcal{N}$ directly to the action selection during environment interaction.
- The original paper utilized Ornstein-Uhlenbeck noise, which creates temporally correlated, mean-reverting patterns that mimic inertial drift.

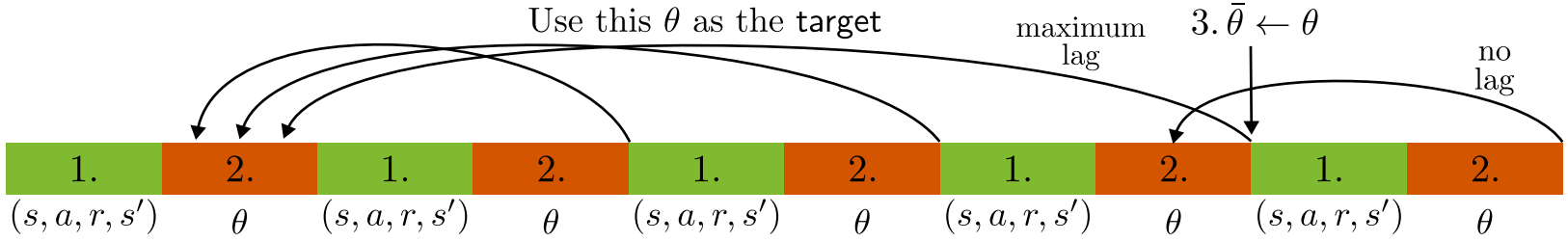
4. **Transition to "soft updates" for target networks**

- In DQN, target network weights are periodically copied exactly from the online network every few thousand steps.
- For continuous actor-critic configurations, hard updates changed the value landscape too abruptly.
- Instead: soft update, where target networks track the online networks smoothly at every single training step using an interpolation factor $\tau \ll 1$ (e.g., $\tau = 0.001$):

$$\bar{\theta} \leftarrow \tau\theta + (1-\tau)\bar{\theta}, \qquad \bar{\phi} \leftarrow \tau\phi + (1-\tau)\bar{\phi}.$$

  $\Rightarrow$ targets change slowly, providing an unmoving baseline that stabilizes the deep network's gradients.

- We have seen this before under *Polyak averaging*.



Twin Delayed Deep Deterministic Policy Gradient (TD3)

Twin Delayed Deep Deterministic Policy Gradient (TD3)

DDPG works on many tasks, but **is highly sensitive** to hyperparameters as well as other randomness (e.g., the sampling).
⇒ In TD3 (Fujimoto, Hoof, and Meger 2018), three main issues were identified and addressed:

1. Maximization bias ⇒ clipped double Q-learning

- Because the target uses a greedy step over actions $a = \mu_\phi(s)$, errors accumulate ⇒ large overestimation bias.
- Two independent critics (Q_{θ_1} / Q_{θ_2} and targets $Q_{\bar{\theta}_1}$ / $Q_{\bar{\theta}_2}$), using the minimum of their predictions to compute the target:

$$y = r + \gamma \min_{i=1,2} Q_{\bar{\theta}_i}(s', \mu_{\bar{\phi}}(s'))$$

2. Inaccurate critic ⇒ delayed policy updates

- If the critic is highly inaccurate, updating the actor based on its gradients is counterproductive.
- Delaying the policy updates ensures the critic has reached a reliable value before the actor uses it.

⇒ Update actor (ϕ) and targets ($\bar{\phi}, \bar{\theta}_1, \bar{\theta}_2$) less frequently than critics (θ_1, θ_2), e.g., every $n_{\text{up}} = 2$ steps.

3. Exploitation of artifacts in Q ⇒ target action smoothing

- Deterministic policies are prone to exploiting sharp peaks or artifacts in the Q-function.
- Adding noise smooths out the value landscape, ensuring that similar actions yield similar values.

⇒ TD3 adds a small amount of clipped noise (clipping constant c) to the target action before feeding it into the target critic:

$$\tilde{a} = \mu_{\bar{\phi}}(s) + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}^2), -c, c).$$

The TD3 algorithm

1. **Interact**: Sample $\{s_t, a_t, r_t, s_{t+1}\}$ using $a_t = \mu_\phi(s_t) + \epsilon$ ($\epsilon \sim \mathcal{N}\left(0, \sigma^2\right)$) and store in the replay buffer $\mathcal{D}$.
2. **Sample**: Draw random mini-batch of N transitions: $\mathcal{B} \subset \mathcal{D}$.
3. **Update critic**: Calculate targets y_i by minimizing over two target networks (💡 the **Twin**):

$$y_i = r_i + \gamma \min_{j \in \{1,2\}} Q_{\bar{\theta}_j}(s_{i+1}, \tilde{a}_{i+1}), \qquad \text{with noisy action } \tilde{a}_{i+1} = \mu_{\bar{\phi}}(s_{i+1}) + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}^2), -c, c).$$

The two critics are updated by minimizing the Bellman errors:

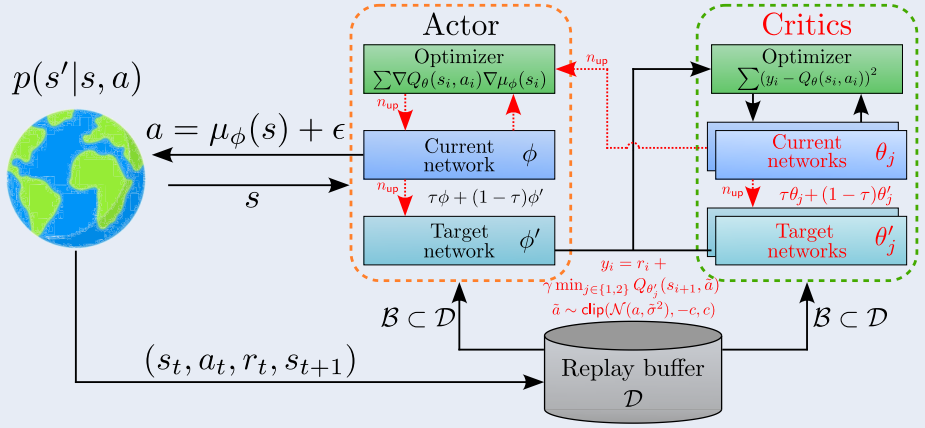
$$L_Q(\theta_j) = \frac{1}{N} \sum_i \left(y_i - Q_{\theta_j}(s_i, a_i)\right)^2, \qquad \theta_j \leftarrow \theta_j + \alpha_\theta \frac{2}{N} \sum_{i=1}^{N} \left(y_i - Q_{\theta_j}(s_i, a_i)\right) \nabla_\theta Q_{\theta_j}(s_i, a_i).$$

Perform next steps only every n_{up} steps: (💡 the **Delayed**):

4. **Update actor**:

$$\phi \leftarrow \phi + \alpha \frac{1}{N} \sum_{i=1}^{N} \nabla_a Q_{\theta_1}(s_i, a)\Big|_{a=\mu_\phi(s_i)} \nabla_\phi \mu_\phi(s_i).$$

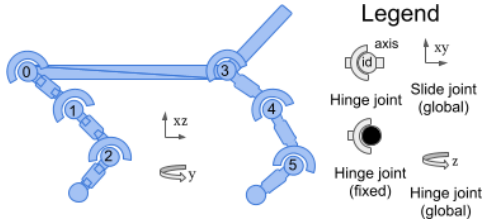
5. **Soft updates**: Incremental target updates ($\bar{\phi}$ / $\bar{\theta}_1$ / $\bar{\theta}_2$).

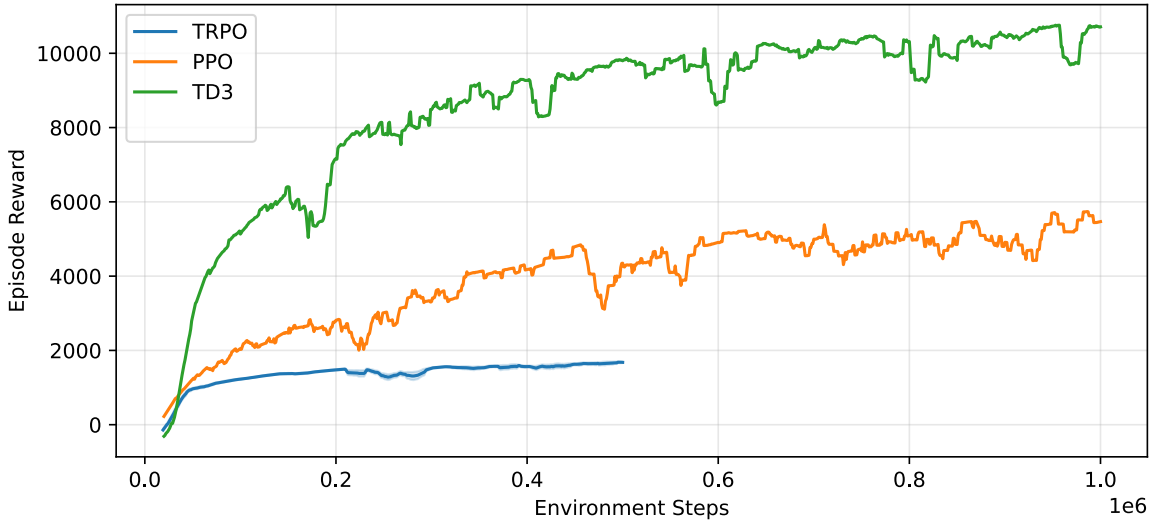


Example: Half-Cheetah

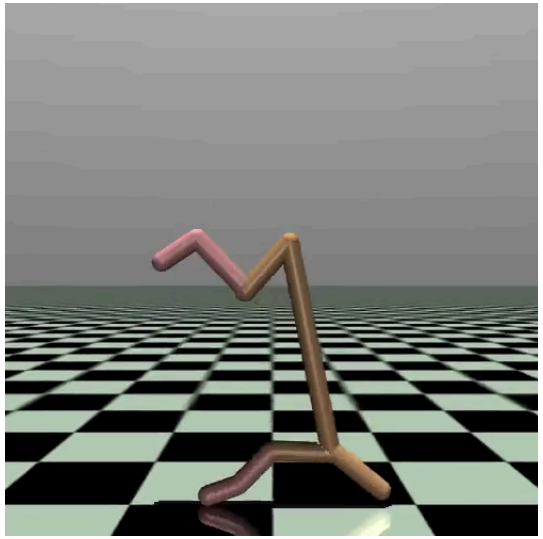
Half Cheetah from the MuJoCo library.

- $\mathcal{S}$: 17 states, $\mathcal{A}$: 6 actions.
- Reward: Forward-Cost - Control-Cost.

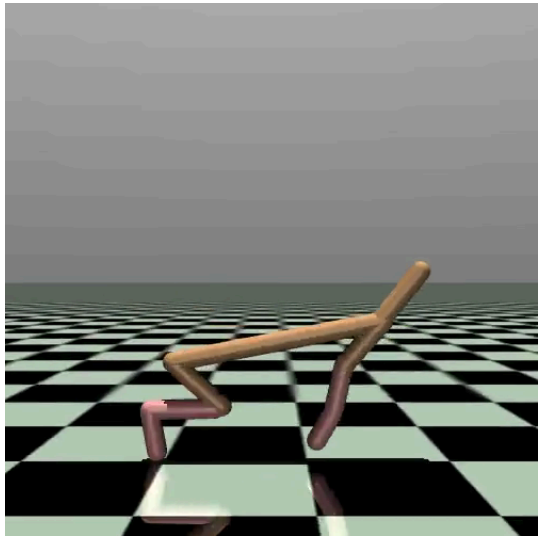




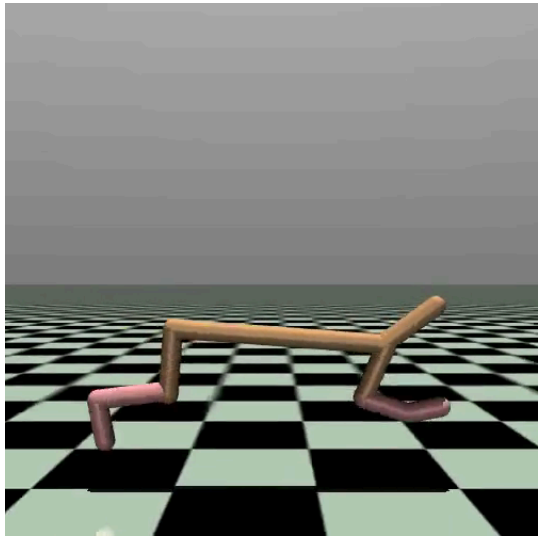
Results: TRPO vs. PPO vs. TD3 (from the RL Baselines3 Zoo)



TRPO



PPO



TD3

Soft actor-critic (SAC)

Introducing entropy

The limitations of both DDPG and TD3 are still quite severe.

- Overestimation bias (DDPG).
- Hyperparameter sensitivity (DDPG).
- Lack of exploration (both).
- Sample inefficiency due to Local Optima (both).

Entropy (defined by Claude Shannon in 1948).

- SAC (Haarnoja, Zhou, Abbeel, et al. 2018) introduces an **entropy term** in the loss function.

$$L_\pi(\phi) = \sum_{t=0}^{T-1} \gamma^t \mathbb{E}_{(s_t,a_t)\sim\rho_{\pi_\phi}} \left[r_t + \alpha \mathcal{H}(\pi_\phi(\cdot|s_t))\right], \quad \text{where } \mathcal{H}(\pi_\phi(\cdot|s)) = \mathbb{E}_{a\sim\pi_\phi(\cdot|s)} \left[-\log \pi_\phi(a|s)\right]. \tag{3}$$

- It describes the level of uncertainty (or unpredictability) of a random variable.
- $\mathcal{H}(\pi_\phi(\cdot|s_t))$ is the entropy, measuring how unpredictable the policy is.
  High entropy $\Rightarrow$ the agent explores widely. **Low entropy** $\Rightarrow$ it is focused on a few actions.
- The *temperature* α (a tunig parameter) determines how much the agent values exploration vs. exploitation.
- By rewarding the agent for being unpredictable, it
  - naturally explores the entire environment,
  - prevents the policy from collapsing into a single repetitive action too early,
  - becomes more robust to environment changes.

The entropy objective and value function

- Let's look at the definition of the entropy in Eq. (3):

$$\mathcal{H}(\pi(\cdot|s)) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[-\log \pi\left(a|s\right)\right].$$

- Now let's expand the value function in the RL objective (3):

$$\begin{aligned} V^{\pi}(s_t) &= \mathbb{E}_{(s_t,a_t)\sim\rho_{\pi_\phi}}\left[\sum_{k=0}^{\infty}\gamma^k(r_{t+k} + \alpha\mathcal{H}(\pi_\phi(\cdot|s_{t+k})))\right] \\ &= \mathbb{E}_{a_t\sim\pi(\cdot|s_t)}\left[r_t + \alpha\mathcal{H}(\pi_\phi(\cdot|s_t))\right] + \mathbb{E}_{(s_t,a_t)\sim\rho_{\pi_\phi}}\left[\sum_{k=1}^{\infty}\gamma^k(r_{t+k} + \alpha\mathcal{H}(\pi_\phi(\cdot|s_{t+1})))\right] \\ &= \mathbb{E}_{a_t\sim\pi(\cdot|s_t)}\left[r_t - \alpha\log\pi(a_t|s_t)\right] + \gamma\mathbb{E}_{s_{t+1}\sim p(s_{t+1})}\left[V^{\pi}(s_{t+1})\right] \end{aligned} \tag{4}$$

Deriving the entropy Q-function

- As a_t is already chosen, there is no entropy for the immediate action (its probability is one after selection).
- The entropy only applies to future actions.
- Therefore, we define the **soft Q-function** as the immediate reward plus the discounted future soft values:

$$Q^\pi(s_t, a_t) = r_t + \gamma \mathbb{E}_{s_{t+1} \sim p(s_{t+1})} \left[V^\pi(s_{t+1}) \right]. \tag{5}$$

- Substitute (5) into (4):

$$V^\pi(s_t) = \mathbb{E}_{a_t \sim \pi(\cdot|s_t)} \Big[\underbrace{r_t + \gamma \mathbb{E}_{s_{t+1} \sim p(s_{t+1})} [V^\pi(s_{t+1})]}_{= Q^\pi(s_t, a_t) \text{ (Eq. (5))}} - \alpha \log \pi(a_t|s_t) \Big] = \mathbb{E}_{a_t \sim \pi(\cdot|s_t)} \Big[Q^\pi(s_t, a_t) - \alpha \log \pi(a_t|s_t) \Big].$$

- We thereby obtain the **soft Bellman equation**:

$$Q^\pi(s_t, a_t) = r_t + \gamma \mathbb{E}_{(s_{t+1}, a_{t+1}) \sim \rho_{\pi_\phi}} \left[Q^\pi(s_{t+1}, a_{t+1}) - \alpha \log \pi(a_{t+1}|s_{t+1}) \right]. \tag{6}$$

Soft actor-critic: ingredients

Neural networks

- An *actor network* π_ϕ.
- Two *critic networks* Q_{θ_1} / Q_{θ_2} and their respective *target networks* $Q_{\bar{\theta}_1}$ / $Q_{\bar{\theta}_2}$.

Objective functions

- Critic updates (based on (6)):

$$L_Q(\theta_j) = \frac{1}{N}\sum_{i=1}^{N}\left(y_i - Q_{\theta_j}(s_i, a_i)\right)^2 \quad \text{with target} \quad y_i = r_i + \gamma\left(\min_{j=1,2} Q_{\bar{\theta}_j}(s_i', a_i') - \alpha \log \pi_\phi(a_i'|s_i')\right). \tag{7}$$

- Actor updates (where we either use θ_1 or the minimum of both critics):

$$L_\pi(\phi) = \frac{1}{N}\sum_{i=1}^{N}\left(\alpha \log \pi_\phi(a_i|s_i) - Q_{\theta_1}(s_i, a_i)\right). \tag{8}$$

- Temperature updates:

$$L(\alpha) = \frac{1}{N}\sum_{i=1}^{N}\left(-\alpha \log \pi_\phi(a_i|s_i) - \alpha\bar{\mathcal{H}}\right). \tag{9}$$

The actor objective (1)

- To see how we arrived at the objective L_π, we start from defining a target distribution over the actions.
- One way to represent the target distribution: probability of choosing an action proportional to its exponential soft Q-value:

$$\pi_{\text{target}}(a|s) = \frac{\exp\left(\frac{Q^\pi(s,a)}{\alpha}\right)}{Z^\pi(s)} \qquad \text{with} \quad Z^\pi(s) = \int \exp\left(\frac{Q^\pi(s,a)}{\alpha}\right) \mathrm{d}a.$$

- This is known as the **Boltzmann distribution** (or **Gibbs distribution**), with two useful properties:
  - Actions with higher Q-values have exponentially higher probabilities of being chosen.
  - The temperature α scales how pronounced these differences are. If $\alpha \to 0$, it becomes a sharp peak at the max Q-value (greedy). If $\alpha \to \infty$, it becomes a uniform distribution (pure random exploration).
- The SAC goal is thus to push the policy towards the target distribution:

$$\begin{aligned} L_\pi(\phi) &= \bar{D}_{\text{KL}}\left(\pi_\phi \| \pi_{\text{target}}\right) = \mathbb{E}_{s\sim\rho, a\sim\pi_\phi}\left[\log\left(\frac{\pi_\phi(a|s)}{\pi_{\text{target}}(a|s)}\right)\right] = \mathbb{E}_{s\sim\rho, a\sim\pi_\phi}\left[\log\left(\pi_\phi(a|s)\right) - \log\left(\frac{\exp\left(\frac{Q^\pi(s,a)}{\alpha}\right)}{Z^\pi(s)}\right)\right] \\ &= \mathbb{E}_{s\sim\rho, a\sim\pi_\phi}\left[\log\left(\pi_\phi(a|s)\right) - \log\left(\exp\left(\frac{Q^\pi(s,a)}{\alpha}\right)\right) + \log\left(Z^\pi(s)\right)\right] \\ &= \mathbb{E}_{s\sim\rho, a\sim\pi_\phi}\left[\log\left(\pi_\phi(a|s)\right) - \frac{Q^\pi(s,a)}{\alpha} + \log\left(Z^\pi(s)\right)\right]. \end{aligned}$$

The actor objective (2)

$$L_\pi(\phi) = \mathbb{E}_{s\sim\rho, a\sim\pi_\phi}\left[\log\left(\pi_\phi(a|s)\right) - \frac{Q^\pi(s,a)}{\alpha} + \log\left(Z^\pi(s)\right)\right].$$

- Multiplying by α and neglecting the policy-independent Z-term don't change the minimizer $\Rightarrow$ Sampling version:

$$L_\pi(\phi) = \frac{1}{N}\sum_{i=1}^{N}\left(\alpha\log\pi_\phi(a_i|s_i) - Q_{\theta_1}(s_i,a_i)\right). \tag{8}$$

Soft Policy Improvement Theorem (Haarnoja, Zhou, Hartikainen, et al. 2018, Lemma 2)
If we define:

$$\pi_{\text{new}} = \arg\min_{\pi'} D_{\text{KL}}\left(\pi'(\cdot|s)\middle\|\frac{\exp\left(\frac{Q^{\pi_{\text{old}}}(\cdot|s)}{\alpha}\right)}{Z^{\pi_{\text{old}}}(s)}\right),$$

then the soft Q-value of the new policy is guaranteed to be monotonically greater than or equal to the old one for all (s,a):

$$Q^{\pi_{\text{new}}}(s,a) \geq Q^{\pi_{\text{old}}}(s,a) \quad \forall(s,a).$$

The reparametrization trick

- Because the action a is sampled from a stochastic policy π_ϕ, we cannot backpropagate gradients from the critic through the action to the actor directly.
- Fix in SAC $\Rightarrow$ the **reparameterization trick**: We express the action as a deterministic function of the state and an independent noise vector ϵ:

$$a = f_\phi(\epsilon; s) = \tanh(\mu_\phi(s) + \sigma_\phi(s) \odot \epsilon), \quad \epsilon \sim \mathcal{N}(0, I).$$

  - The neural network deterministically outputs the mean $\mu_\phi(s)$ and standard deviation $\sigma_\phi(s)$.
  - The $\tanh$ function bounds the actions to a valid range (e.g., $[-1, 1]$).
- Rewriting the objective with this reparameterization allows us to rewrite the expectation over the noise ϵ:

$$L_\pi(\phi) = \mathbb{E}_{s \sim \mathcal{D}, \epsilon \sim \mathcal{N}} [\alpha \log \pi_\phi(f_\phi(\epsilon; s)|s) - Q_{\phi_1}(s, f_\phi(\epsilon; s))].$$

- Now, the gradient with respect to ϕ can flow through f_ϕ.
- Analogous to earlier (deterministic) policy gradients, we obtain (Haarnoja, Zhou, Hartikainen, et al. 2018):

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{s \sim \mathcal{D}, \epsilon \sim \mathcal{N}} [\nabla_\phi \alpha \log \pi_\phi(a|s) + (\alpha \nabla_a \log \pi_\phi(a|s) - \nabla_a Q_{\theta_1}(s, a)) \nabla_\phi f_\phi(\epsilon; s)].$$

💡 Because we pass the Gaussian sample through a $\tanh$ function, the log-likelihood must be corrected using the Jacobian of the transformation: $\log \pi_\theta(a|s) = \log \mu(u|s) - \sum_{i=1}^{m} \log(1 - \tanh^2(u_i))$, where u is the pre-tanh action value.

The temperature objective

- To prevent hand-tuning the entropy coefficient α, SAC treats it as a constrained optimization problem: maximize expected reward subject to a minimum entropy constraint $\bar{\mathcal{H}}$ (More details in (Haarnoja, Zhou, Hartikainen, et al. 2018, Chapter 5)).
- SAC tunes α to match a target entropy $\bar{\mathcal{H}}$ (usually chosen as the negative action space dimension $(-m)$).
- The *dual objective function* minimized to find the optimal scaling parameter α is:

$$L(\alpha) = \mathbb{E}_{s\sim\mathcal{D},a\sim\pi_\theta}\left[-\alpha \log \pi_\theta(a|s) - \alpha\bar{\mathcal{H}}\right] = \mathbb{E}_{s\sim\mathcal{D},a\sim\pi_\theta}\left[-\alpha(\log \pi_\theta(a|s) + \bar{\mathcal{H}})\right]. \tag{9}$$

- Since $L(\alpha)$ is linear with respect to α, taking the derivative is straightforward:

$$\nabla_\alpha L(\alpha) = -\mathbb{E}_{s\sim\mathcal{D},a\sim\pi_\theta}\left[(\log \pi_\theta(a|s) + \bar{\mathcal{H}})\right].$$

Intuition of the α Gradient

- If the current policy's entropy is low, then $-\log \pi_\phi(a|s)$ will be a large positive number.
  - If $-\log \pi_\phi(a|s) > \bar{\mathcal{H}}$, the gradient is negative, which causes α to increase during gradient descent ($\alpha \leftarrow \alpha - \lambda\nabla_\alpha L(\alpha)$).
  - Higher α forces the actor to prioritize exploration.
- If the policy's entropy is high, the gradient becomes positive, causing α to decrease, shifting priorities toward maximizing standard environmental rewards.

The SAC algorithm

Algorithm 1 Soft Actor-Critic

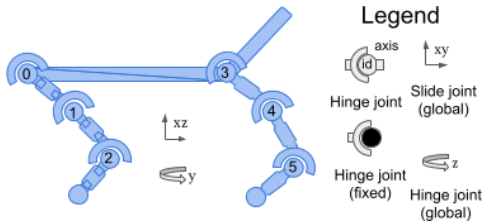
Input: θ_1, θ_2, ϕ ▷ Initial parameters
$\bar{\theta}_1 \leftarrow \theta_1, \bar{\theta}_2 \leftarrow \theta_2$ ▷ Initialize target network weights
$\mathcal{D} \leftarrow \emptyset$ ▷ Initialize an empty replay pool
for each iteration **do**
  for each environment step **do**
    $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t|\mathbf{s}_t)$ ▷ Sample action from the policy
    $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ ▷ Sample transition from the environment
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$ ▷ Store the transition in the replay pool
  end for
  for each gradient step **do**
    $\theta_i \leftarrow \theta_i - \lambda_Q \nabla_{\theta_i} L_Q(\theta_i)$ for $i \in \{1, 2\}$ ▷ Update the Q-function parameters
    $\phi \leftarrow \phi - \lambda_\pi \nabla_\phi L_\pi(\phi)$ ▷ Update policy weights
    $\alpha \leftarrow \alpha - \lambda \nabla_\alpha L(\alpha)$ ▷ Adjust temperature
    $\bar{\theta}_i \leftarrow \tau\theta_i + (1-\tau)\bar{\theta}_i$ for $i \in \{1, 2\}$ ▷ Update target network weights
  end for
end for
Output: θ_1, θ_2, ϕ ▷ Optimized parameters

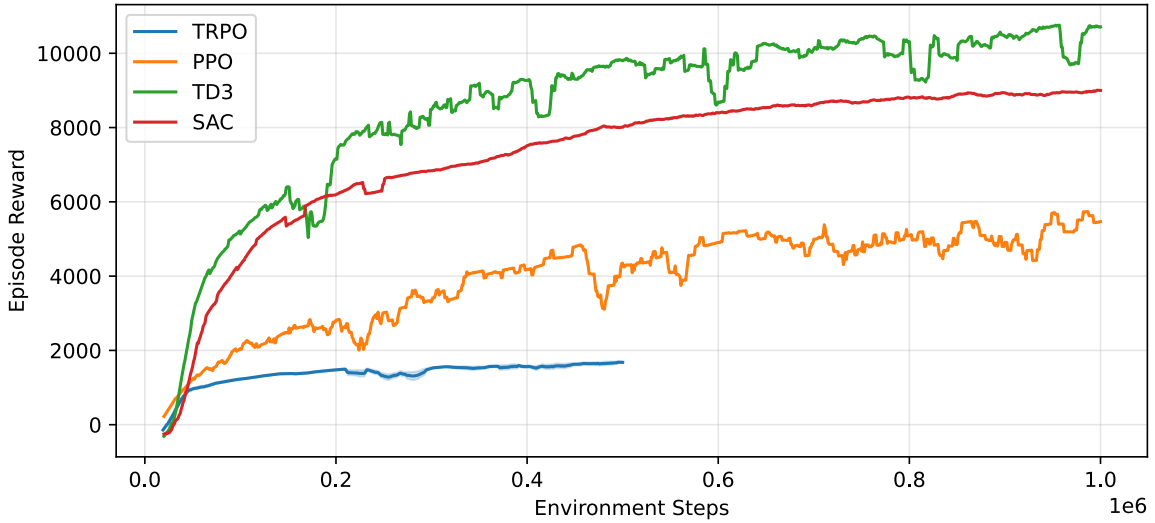
(*Haarnoja, Zhou, Hartikainen, et al. 2018*)

Example: Half-Cheetah

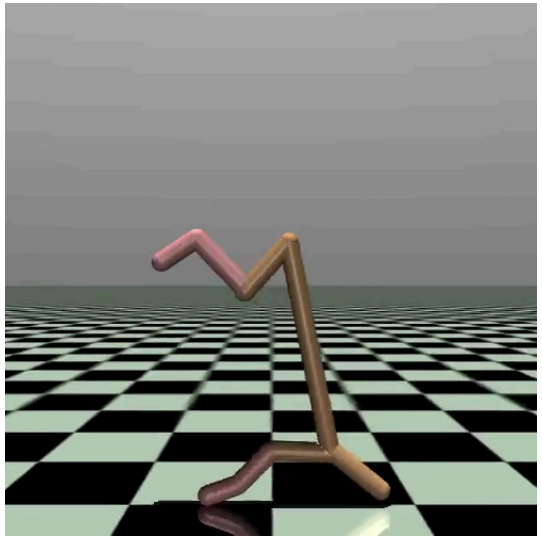
Half Cheetah from the MuJoCo library.

- $\mathcal{S}$: 17 states, $\mathcal{A}$: 6 actions.
- Reward: Forward-Cost - Control-Cost.

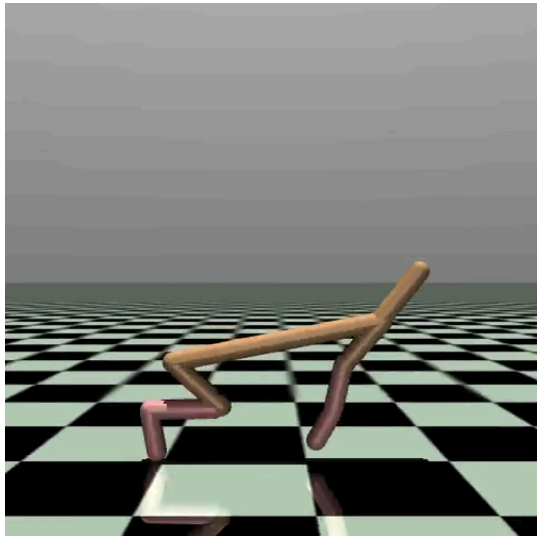




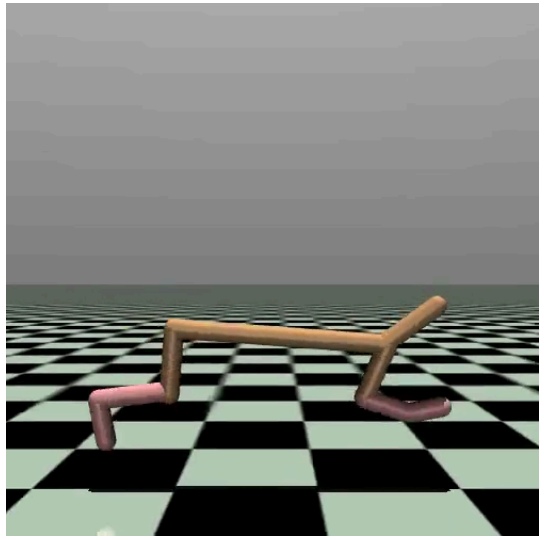
Results: TRPO vs. PPO vs. TD3 vs. SAC (from the RL Baselines3 Zoo)



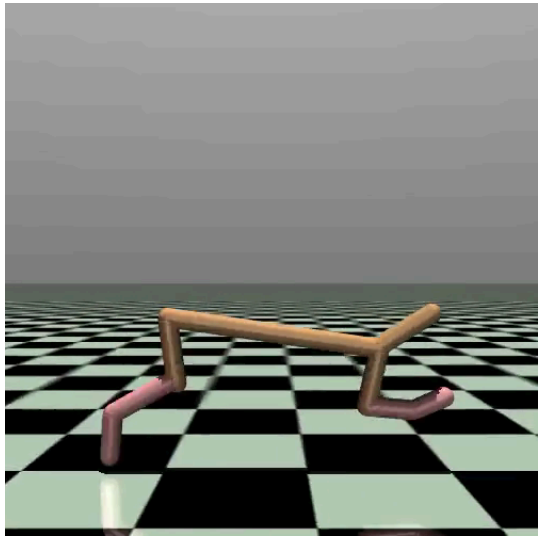
TRPO



PPO

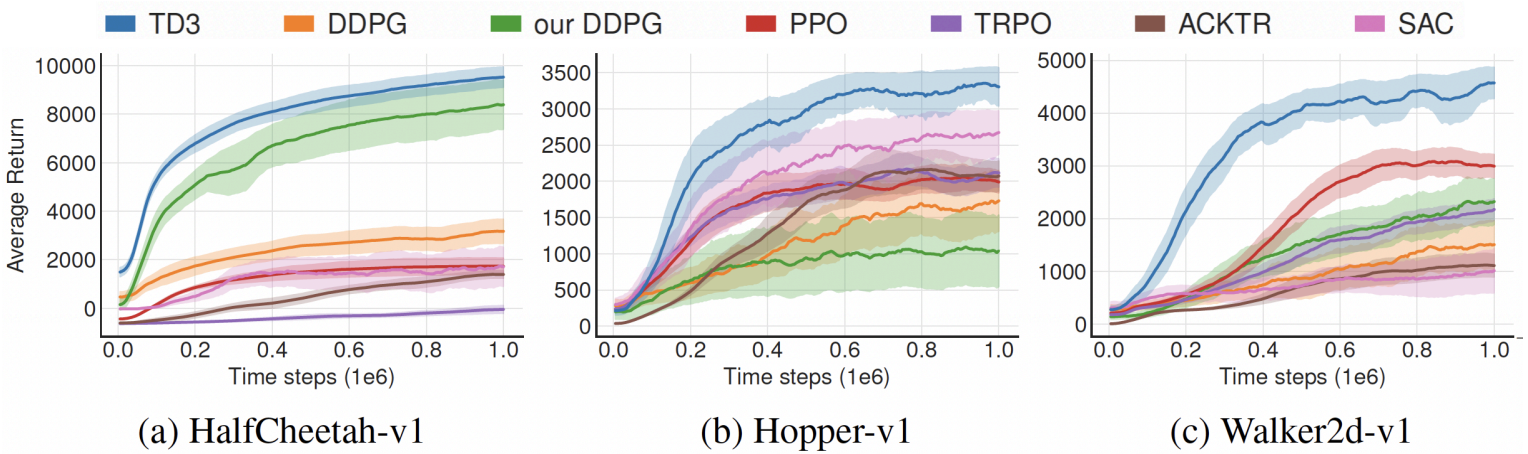


TD3



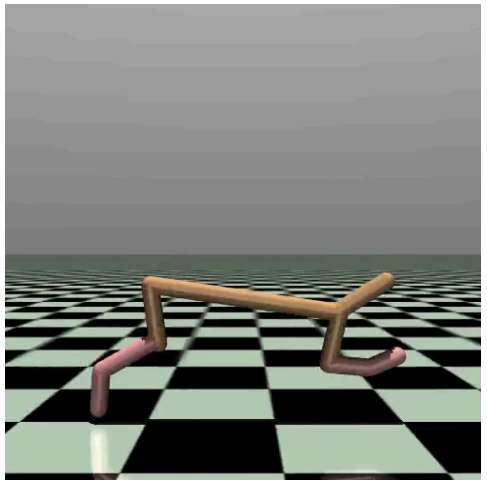
SAC

Example: Some MuJoCo environments

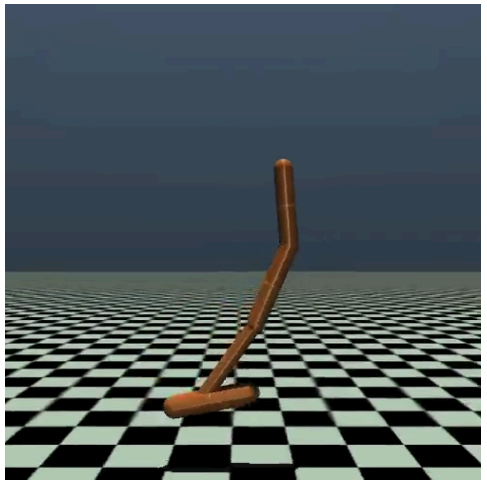


(a) HalfCheetah-v1 (b) Hopper-v1 (c) Walker2d-v1

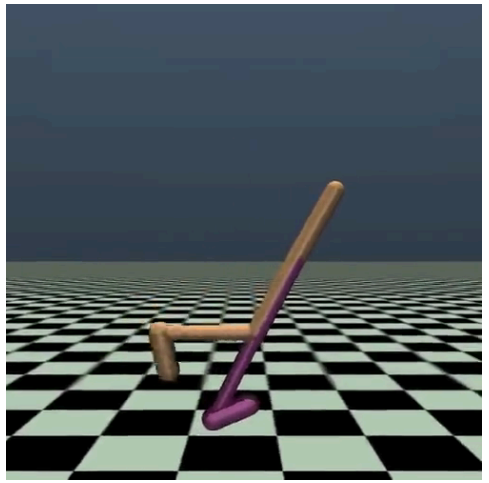
Source: (Fujimoto, Hoof, and Meger 2018, Figure 5).



Half cheetah (SAC).



Hopper (SAC).



Walker 2D (SAC).

💡 Trained models from StableBaselines3.

Notes

- The SAC results *reported in the TD3 paper* (Fujimoto, Hoof, and Meger 2018) (see left) are worse than the TD3 results.
- Reason: there were two versions of SAC in quick succession.
  1. The now-standard that we have discussed (Haarnoja, Zhou, Hartikainen, et al. 2018). It also considered some ideas from the TD3 paper such as twin Q networks to avoid maximization bias.
  2. The original one (Haarnoja, Zhou, Abbeel, et al. 2018) went out of fashion quickly, but appeared around the same time as TD3. It is thus likely that (Fujimoto, Hoof,

and Meger 2018) compared against this version.

Comparison of the algorithms we have seen in part II

Comparison of the algorithms we have seen in part II

- Actor-Critic.
- DDPG $\Rightarrow$ variance reduction via deterministic policy.
- TD3 $\Rightarrow$ addresses maximization bias (twin networks) and inaccurate critics (via delayed updates).
- SAC $\Rightarrow$ entropy objective for natural exploration

Feature	DDPG	TD3	SAC
Policy Type	Deterministic	Deterministic	Stochastic
Exploration	Additive Noise (e.g., OU Noise)	Additive Noise (Gaussian)	Intrinsic (Entropy Maximization)
Q-Overestimation Cure	None	Clipped Double-Q	Clipped Double-Q
Sensitivity to Hyperparameters	Extremely High	High	Low (Very Stable)
Sample Efficiency	Moderate	Good	Excellent

References

Abdelwanis, Ali, Barnabas Haucke-Korber, Darius Jakobeit, Wilhelm Kirchgässner, Marvin Meyer, Maximilian Schenke, Hendrik Vater, Oliver Wallscheid, and Daniel Weber. 2026. “Reinforcement Learning: A Comprehensive Open-Source Course.” *Journal of Open Source Education* 9 (97). The Open Journal: 306. doi:10.21105/jose.00306.

Degris, Thomas, Martha White, and Richard S. Sutton. 2012. “Off-Policy Actor-Critic.” In *International Conference on Machine Learning*.

Fujimoto, Scott, Herke van Hoof, and David Meger. 2018. “Addressing Function Approximation Error in Actor-Critic Methods.” In *Proceedings of the 35th International Conference on Machine Learning*, edited by Jennifer Dy and Andreas Krause, 80:1587–96. Proceedings of Machine Learning Research. PMLR.

Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.” In *Proceedings of the 35th International Conference on Machine Learning*, edited by Jennifer Dy and Andreas Krause, 80:1861–70. Proceedings of Machine Learning Research. PMLR.

Haarnoja, Tuomas, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, et al. 2018.